

ROS2 for ATOSFleetManagement Cheat Sheet

1) Source environment

```
```bash
source /opt/ros/humble/setup.bash
source ~/atos_ws/install/setup.bash
```
```

2) Build ATOSFleetManagement packages

```
```bash
cd ~/atos_ws
colcon build --packages-select atos atos_gui --symlink-install
source install/setup.bash
```
```

3) Start ATOSFleetManagement (normal)

```
```bash
ros2 launch atos launch_atosfleetmanagement.py insecure:=True
```
```

4) Start ATOSFleetManagement with 3 simulators

```
```bash
ros2 launch atos launch_atosfleetmanagement.py insecure:=True with_truck_simulator:=
True
```
```

Default simulator setup in launch

```
- `L5S-TRUCK-SIM-1`: `start_index=0`, `target_speed_kmh=80`, `ignore_warning_speed_c
ommands=True`
- `L5S-TRUCK-SIM-2`: `start_index=250`, `target_speed_kmh=40`
- `L5S-TRUCK-SIM-3`: `start_index=500`, `target_speed_kmh=40`
```

5) Open GUI

```
- URL: `http://localhost:8420`
- Go to tab: `RuralRoad Map`
- UI shows live trucks + `Distance To Next Truck Ahead`
```

6) Run in Docker (same image, with/without simulators)

```
```bash
cd ~/Documents/repos/ATOS
docker compose -f docker-compose-fleetmanagement.yml up --build
```
```

With simulators:

```
```bash
WITH_TRUCK_SIMULATOR=True docker compose -f docker-compose-fleetmanagement.yml up --
build
```
```

Without simulators:

```
```bash
WITH_TRUCK_SIMULATOR=False docker compose -f docker-compose-fleetmanagement.yml up -
-build
```
```

7) Run Docker as a systemd service

```
```bash
sudo mkdir -p /opt/atos
sudo rsync -a ~/Documents/repos/ATOS/ /opt/atos/
sudo cp /opt/atos/scripts/atosfleetmanagement.env.example /etc/default/atosfleetmanagement
sudo cp /opt/atos/scripts/atosfleetmanagement.service /etc/systemd/system/atosfleetmanagement.service
sudo systemctl daemon-reload
sudo systemctl enable --now atosfleetmanagement
```
```

Service controls:

```
```bash
sudo systemctl status atosfleetmanagement
sudo systemctl restart atosfleetmanagement
sudo journalctl -u atosfleetmanagement -f
```
```

8) Check running nodes

```
```bash
ros2 node list
```
```

9) Check key topics

```
```bash
ros2 topic list | rg "truck_objects|speed_command"
```
```

10) Watch live truck states for GUI

```
```bash
ros2 topic echo /atos/truck_objects/state
```
```

11) Watch control speed commands

```
```bash
ros2 topic echo /atos/truck_objects/speed_command
```
```

12) TruckObjectControl COT interfaces

- ROS topic input: `/atos/truck\_objects/cot` (placeholder format)
- TCP input: `0.0.0.0:8114` listener in TruckObjectControl
- Truck clients connect to: `127.0.0.1:8114` (same machine) or `:8114`
- CoT `` is in `m/s` (GUI displays `km/h`)

13) Placeholder ROS COT test (manual)

```
```bash
ros2 topic pub /atos/truck_objects/cot std_msgs/msg/String "data: 'id=truck_1;distance_m=1000;tcp_connected=1;lat=57.78357;lon=12.76389;speed_mps=2.8;course_deg=150'" - 1
```
```

14) Start one simulator manually (custom test)

```
```bash
ros2 run atos atos_truck_simulator --ros-args \
 -p uid:=L5S-TRUCK-UBUNTU \
```

```
-p start_index:=300 \
-p target_speed_kmh:=35.0 \
-p acceleration_mps2:=0.6 \
-p tcp_host:=127.0.0.1 \
-p tcp_port:=8114
````
```

15) Simulator option: ignore first-limit slowdown

Use this only on selected trucks:

```
````bash  
-p ignore_warning_speed_commands:=true
````
```

Behavior:

- Ignores non-zero warning command (for example `30 km/h` at `< 400 m`)
- Still obeys stop command (`0 km/h` at `< 200 m`)

16) Fleet control thresholds

- Gap `< 400 m` => publish `target\_speed\_kmh=30`
- Gap `< 200 m` => publish `target\_speed\_kmh=0`

17) Stop everything

- Press `Ctrl+C` in launch terminal